

**Course: Fundamentals of Generative AI and Large Language
Models: Theory and Practice**

**Week 2: Autoencoders: The Representation
Learning Model**

Today's Session Overview

- Regularization Techniques in AE: Denoising Autoencoder

NPTEL

Introduction

A basic encoder:

$\hat{h} = g(x) = g(Wx + b)$, where $g(\cdot)$ is non linear activation function

The decoder is:

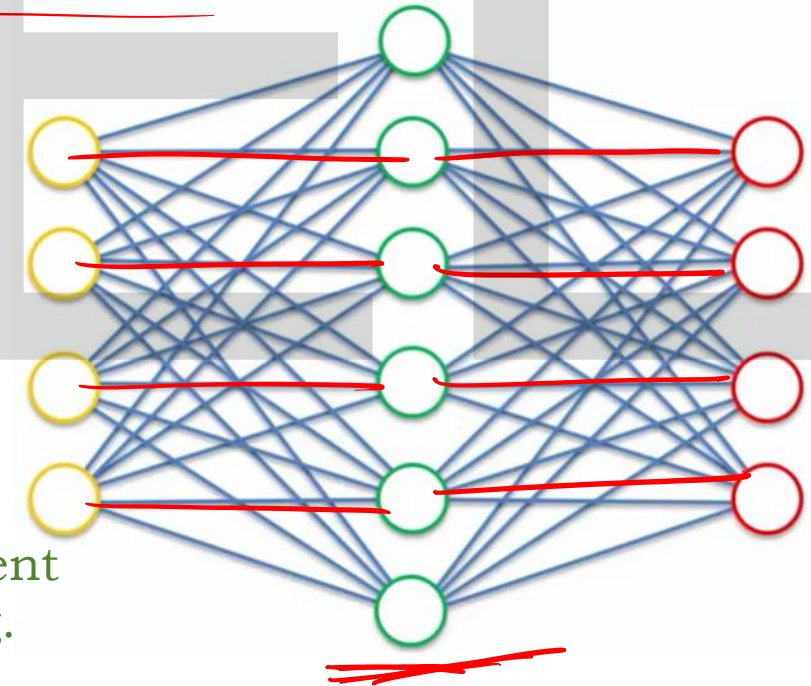
$\hat{x} = f(h) = f(W'h + c)$, where $f(\cdot)$ is the decoder activation

Train the network to $\min Loss(\hat{x}, x)$

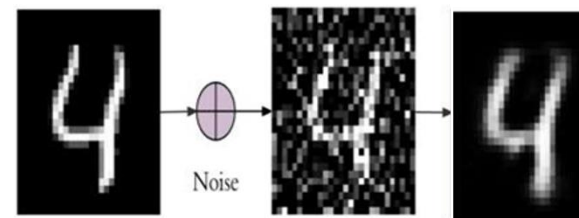
When the hidden layer dimension is large, the model may simply copy the input to the output without learning meaningful features.

In such cases, the autoencoder can learn an approximate identity mapping instead of useful structure. This may lead to overfitting.

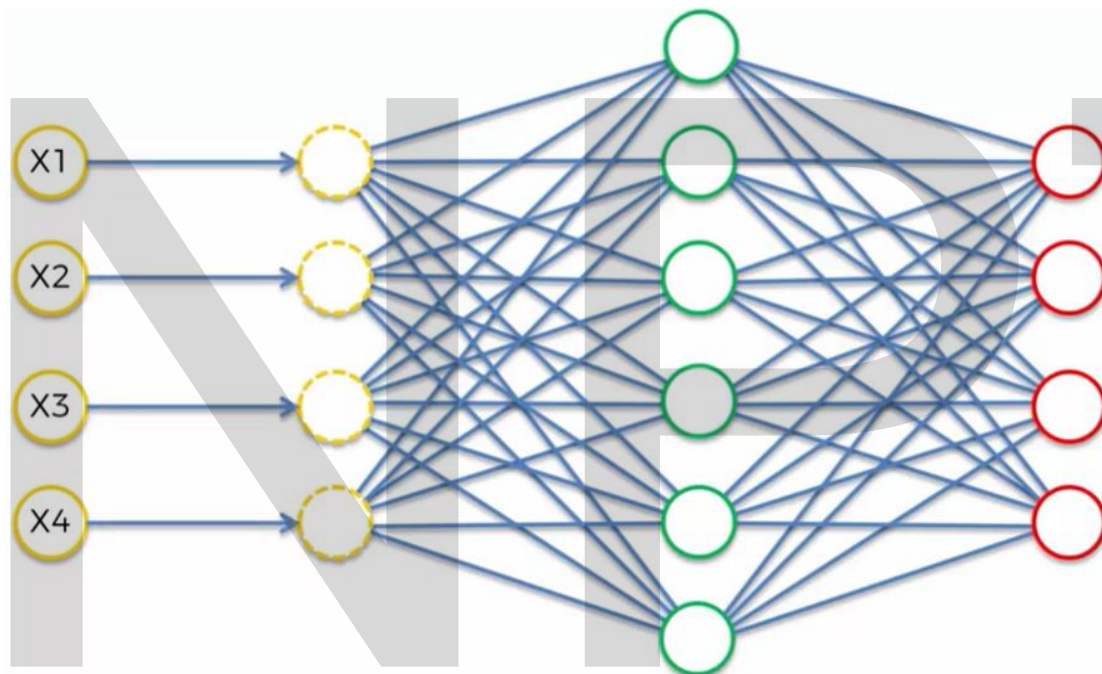
Therefore, a regularized variant of the autoencoder is needed to prevent trivial identity mapping and encourage meaningful feature learning.



Denoising Autoencoder



A denoising autoencoder first corrupts the input randomly using probability-based rule and then learns to reconstruct the original clean input.



corruption
 $x(\text{clean input}) \xrightarrow{\text{corruption}} \tilde{x}(\text{noisy input})$

1. Gaussian noise:

$$\tilde{x} = x + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I)$$

ε is randomly sampled from a Gaussian distribution.

$$x = \begin{bmatrix} 0.50 \\ 0.80 \\ 0.20 \\ 0.90 \end{bmatrix} + \varepsilon = \begin{bmatrix} 0.01 \\ -0.05 \\ 0.03 \\ -0.08 \end{bmatrix} = \begin{bmatrix} 0.51 \\ 0.75 \\ 0.23 \\ 0.82 \end{bmatrix}$$

2. Masking noise: Some components are randomly set to zero.

If there are 5 values and $q = 0.4$, then approximately 2 values are expected to be masked on average.

Example:

$$x = [0.8, 0.3, 0.9, 0.5, 0.2]$$

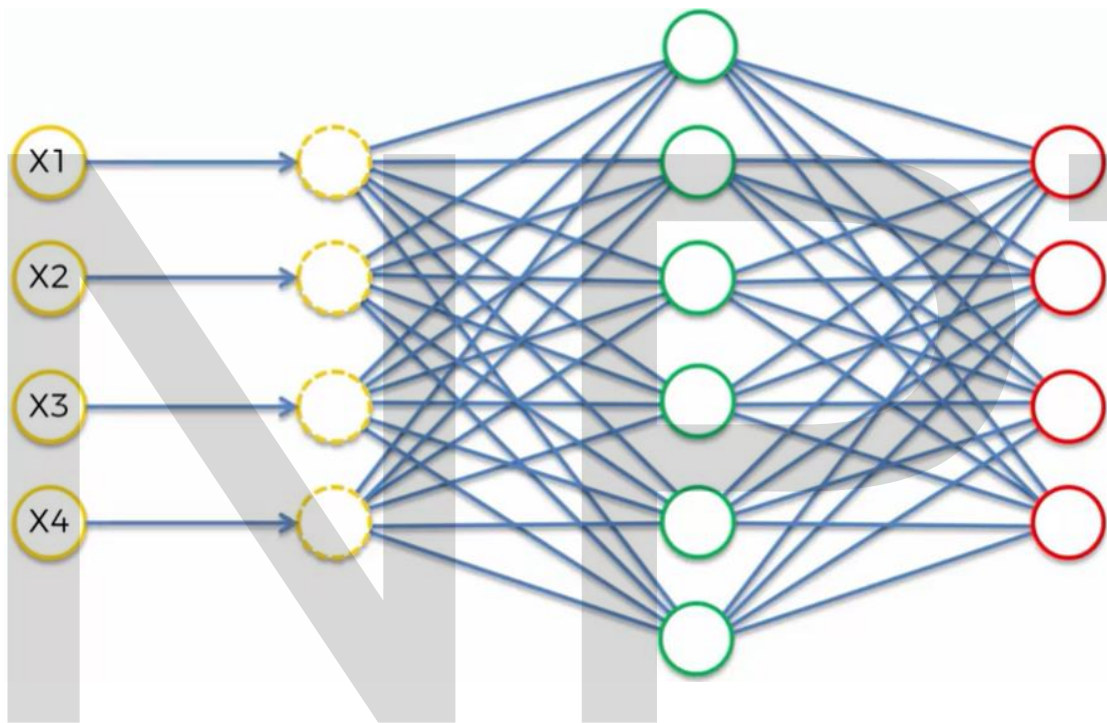
One possible masking result with $q = 0.4$ is:

$$\tilde{x} = [0.8, 0, 0.9, 0, 0.2]$$

$$\tilde{x} = \begin{cases} x_{i,j} = 0 & \text{with probability } q \\ x_{i,j} & \text{with the probability } 1 - q \end{cases}$$

When $q = 0.4$, every component is checked independently each one has a 40% chance of becoming 0 each one has a 60% chance of staying unchanged.

Denoising Autoencoder



3. Salt and pepper noise: In salt-and-pepper noise, a few randomly chosen pixels are corrupted by replacing them with extreme intensity values. Some pixels become completely black and some become completely white, even if they originally had different values.

If the image is binary:

$$x = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

after salt-and-pepper noise,
some pixels may flip
randomly:

$$\tilde{x} = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

So a few pixels are randomly
changed to the opposite
extreme.

If the image is grayscale:

$$x = \begin{bmatrix} 120 & 125 & 130 \\ 123 & 128 & 126 \\ 121 & 124 & 129 \end{bmatrix}$$

✓ some pixel becomes 0
✓ some pixel becomes 255
Then the corrupted image
becomes:

$$\tilde{x} = \begin{bmatrix} 120 & 125 & 255 \\ 123 & 0 & 126 \\ 121 & 124 & 129 \end{bmatrix}$$

Here:

- ✓ pixel 130 changed to 255
→ salt (white)
- ✓ pixel 128 changed to 0 →
pepper (black)

Denoising Autoencoders: Learning Neighboring Pixel Dependencies



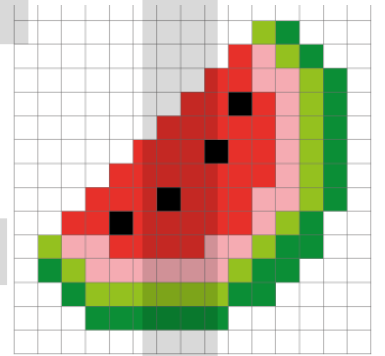
Corrupted input image $\tilde{x}$

In an image, the value of a pixel is usually **not independent** of the values of its neighboring pixels. Neighboring pixels are often **highly correlated**.

$x_{i,j}$

denotes the pixel value at spatial location (i,j) in the image

- In a binary image, $x_{i,j} \in \{0,1\}$
- In a grayscale image, $x_{i,j}$ is an intensity value in $[0, 255]$, or normalized to $[0,1]$



For example, if a pixel $x_{i,j}$ belongs to a curved black stroke, then its nearby pixels are also likely to belong to the same stroke.

$x_{i,j}$ is conditionally dependent on its local neighborhood $\mathcal{N}_{i,j}$

$$\mathcal{N}_{i,j} = \begin{bmatrix} x_{i-1,j-1} & x_{i-1,j} & x_{i-1,j+1} \\ x_{i,j-1} & \textcircled{?} & x_{i,j+1} \\ x_{i+1,j-1} & x_{i+1,j} & x_{i+1,j+1} \end{bmatrix}$$

If the centre pixel is corrupted or missing, its value can be inferred from the surrounding 8 pixels and the local image structure.

Given the neighboring pixels, what is the probability distribution of the centre pixel value?

$$p(\underline{x_{i,j}} | \mathcal{N}_{i,j})$$

Consider a 3×3 neighborhood
Suppose the centre pixel is missing:

$$\mathcal{N}_{i,j} = \begin{bmatrix} 1 & 1 & 0 \\ 1 & ? & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

----->

Assume this is a binary image, where:

- 1 = black pixel
- 0 = background pixel

We want to infer the missing centre pixel from its neighbors.

(2) One more example:

$$\mathcal{N}_{i,j} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & ? & 0 \\ 0 & 1 & 0 \end{bmatrix} \quad \begin{array}{l} \text{Number of 0s} = 6 \\ \text{Number of 1s} = 2 \end{array}$$

So:

$$P(x_{i,j} = 0 \mid \mathcal{N}_{i,j}) = \frac{6}{8} = 0.75 \quad \checkmark$$

$$P(x_{i,j} = 1 \mid \mathcal{N}_{i,j}) = \frac{2}{8} = 0.25$$

Hence:

$$x_{i,j} = 0$$

$$\begin{bmatrix} 0 & 0 & 1 \\ 0 & \mathbf{0} & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

The 8 surrounding pixels are:

$$\{1, 1, 0, 1, 1, 0, 1, 1\}$$

Now count:

Number of 1s = 6

Number of 0s = 2

A simple neighborhood-based estimate is:

$$P(x_{i,j} = 1 \mid \mathcal{N}_{i,j}) = \frac{6}{8} = 0.75$$

$$P(x_{i,j} = 0 \mid \mathcal{N}_{i,j}) = \frac{2}{8} = 0.25$$

Since:

$$P(x_{i,j} = 1 \mid \mathcal{N}_{i,j}) > P(x_{i,j} = 0 \mid \mathcal{N}_{i,j})$$

we infer:

$$x_{i,j} = 1$$

So the filled patch becomes:

$$\begin{bmatrix} 1 & 1 & 0 \\ 1 & \mathbf{1} & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

Grayscale 3×3 neighborhood

Suppose the centre pixel is missing:

$$\mathcal{N}_{i,j} = \begin{bmatrix} 120 & 125 & 130 \\ 123 & ? & 127 \\ 122 & 124 & 126 \end{bmatrix}$$

Assume this is a grayscale image, where each pixel value lies in $[0,255]$.

Step 1: List the 8 surrounding pixel values

The 8 neighbors are:

$\{120, 125, 130, 123, 127, 122, 124, 126\}$

Step 2: Simple neighborhood-based estimate using average

A simple estimate is the mean of the neighboring pixels:

$$x_{i,j} \approx \frac{120 + 125 + 130 + 123 + 127 + 122 + 124 + 126}{8} = \frac{997}{8} = 124.625 \quad \checkmark$$

So the missing centre pixel can be estimated as:

$$x_{i,j} \approx 125$$

after rounding.

So the filled patch becomes:

$$\begin{bmatrix} 120 & 125 & 130 \\ 123 & \underline{125} & 127 \\ 122 & 124 & 126 \end{bmatrix}$$

In an actual denoising autoencoder, the network does **not explicitly average the 8 neighbours**.

Instead, it learns such local intensity relationships automatically through convolution filters and training.

This means:

- In grayscale images, neighbouring pixels usually have similar intensity values
- So, the missing centre pixel can be estimated from the nearby intensities
- Here, the surrounding values are all around 120–130, so the centre is also likely around that range

During training, the denoising autoencoder learns that:

- Neighboring pixels contain useful information
- Local and contextual structure helps reconstruct the clean image

Hence, denoising autoencoders learn **robust latent features** instead of simply copying the input.

```
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(inputs)
```

This line allows the model to **learn from local neighborhoods** through a 3×3 convolution filter.

Why?

Because a 3×3 convolution means:

for every pixel location, the layer looks at the local patch

$$\begin{bmatrix} x_{i-1,j-1} & x_{i-1,j} & x_{i-1,j+1} \\ x_{i,j-1} & x_{i,j} & x_{i,j+1} \\ x_{i+1,j-1} & x_{i+1,j} & x_{i+1,j+1} \end{bmatrix}$$

So if the centre pixel is missing, the filter still sees the surrounding neighboring pixels.

That is exactly how the model learns local neighbor dependencies.

3	5	1	2	7	4
1	0	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

1	0	1
1	0	1
1	0	1

=

17			

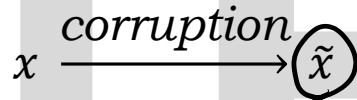
Even if the centre pixel is corrupted, the convolution filter still uses the surrounding neighbouring pixels in the local patch. That is exactly why the model can still learn from the neighbourhood.

Denoising Autoencoders Formulation

A denoising autoencoder receives a corrupted image $\tilde{x}$ as input and learns to reconstruct the original clean image x .

Step 1: Corrupt the clean image:

Start with the original clean image x . A corruption process is applied to obtain a noisy image $\tilde{x}$.



Here:

- x = clean image
- $\tilde{x}$ = corrupted / noisy image

The corrupted image is the input to the denoising autoencoder.

Step 2: Encoder maps corrupted input to latent representation:

The encoder takes the corrupted image $\tilde{x}$ and transforms it into a hidden or latent representation h .

$$\underline{h} = g(\underline{W}\tilde{x} + \underline{b})$$

where:

- W = encoder weight matrix
- b = bias
- $g(\cdot)$ = encoder activation function
- h = latent representation

This latent representation captures the important structure of the corrupted image.

Denoising Autoencoders Formulation

Step 3: Decoder reconstructs the image:

The decoder uses the latent representation h to reconstruct the output image $\hat{x}$

$$\hat{x} = f(W'h + c)$$

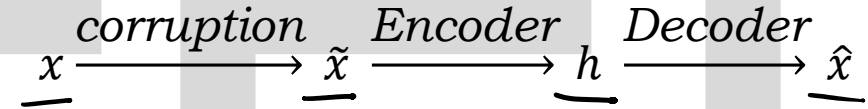
where:

- W' = decoder weight matrix
- c = decoder bias
- $f(\cdot)$ = decoder/output activation function
- $\hat{x}$ = reconstructed image

The goal is that $\hat{x}$ should be as close as possible to the original clean image x .

Step 4: Full denoising autoencoder pipeline:

Combining all the steps:



So the model learns:

$$\tilde{x} \rightarrow \hat{x} \approx x$$

That is, it receives a corrupted image but tries to reconstruct the clean image.

Denoising Autoencoders Formulation

Step 5: Training objective:

The reconstructed image $\hat{x}$ is compared with the original clean image x .

$$\min L(\underline{x}, \underline{\hat{x}})$$

This means the network is trained to minimize the reconstruction loss between the clean image and the reconstructed image.

- **MSE** for continuous-valued data ✓
- **BCE** for binary or normalized images ✓

Step 6: Neighborhood connection:

When a pixel is corrupted or missing, the network cannot recover it from that pixel alone because that value is absent.

So the model must use: *nearby pixels, local image structure, contextual information.*

That is:

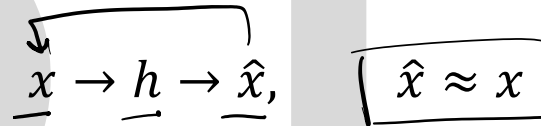
$$\underline{\hat{x}_{i,j}} \approx \hat{F}(\mathcal{N}_{i,j})$$

The reconstructed pixel value at location (i,j) is obtained from some function of its local neighborhood $\underline{\mathcal{N}_{i,j}}$

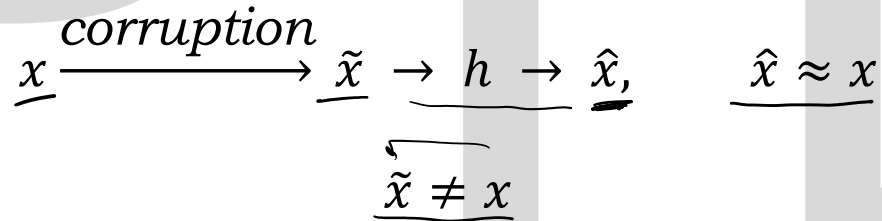
Denoising Autoencoder as a Regularized Variant

A denoising autoencoder is called a **regularized variant** of the standard autoencoder because the input is deliberately corrupted before encoding. This prevents the network from learning a trivial identity mapping.

Standard AE:

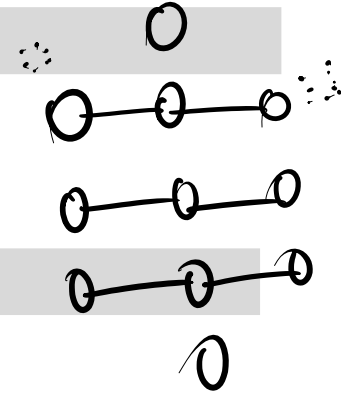


Denoising AE:



Since generally,

simply copying the corrupted input does not minimize the reconstruction loss



Why it provides regularization

- ✓ Corruption removes the shortcut of direct copying
- ✓ The model must infer missing or noisy pixels from context

This encourages learning of robust latent features

Next Session

➤ Sparse Autoencoders

NPTREL

NIPTELL

Thank you